JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class FilterReader

java.lang.Object
    java.io.Reader
        java.io.FilterReader

All Implemented Interfaces:

Closeable, AutoCloseable, Readable

Direct Known Subclasses:

PushbackReader

```
public abstract class FilterReader
extends Reader
```

Abstract class for reading filtered character streams. The abstract class `FilterReader` itself provides default methods that pass all requests to the contained stream. Subclasses of `FilterReader` should override some of these methods and may also provide additional methods and fields.

Since:

1.1

Field Summary

Fields

Modifier and Type	Field	Description
`protected Reader`	**`in`**	The underlying character-input stream.

Fields declared in class Reader

`lock`

Constructor Summary

Constructors

Modifier	Constructor	Description
`protected`	**`FilterReader`**`(Reader in)`	Creates a new filtered reader.

Method Summary

All Methods	Instance Methods	

Modifier and Type	Method	Description
void	close()	Closes the stream and releases any system resources associated with it.
void	mark (int readAheadLimit)	Marks the present position in the stream.
boolean	markSupported()	Tells whether this stream supports the mark() operation.
int	read()	Reads a single character.
int	read(char[] cbuf, int off, int len)	Reads characters into a portion of an array.
boolean	ready()	Tells whether this stream is ready to be read.
void	reset()	Resets the stream.
long	skip(long n)	Skips characters.

Methods declared in class Reader

nullReader, of, read, read, readAllAsString, readAllLines, transferTo

Methods declared in class Object

clone, equals, finalize, getClass, hashCode, notify, notifyAll, toString, wait, wait, wait

Field Details

in

```
protected Reader in
```

The underlying character-input stream.

Constructor Details

FilterReader

```
protected FilterReader(Reader in)
```

Creates a new filtered reader.

Parameters:

`in` - a Reader object providing the underlying stream.

Throws:

`NullPointerException` - if `in` is `null`

Method Details

read

```
public int read()
         throws IOException
```

Reads a single character.

Overrides:

`read` in class `Reader`

Returns:

The character read, as an integer in the range 0 to 65535 (`0x00-0xffff`), or -1 if the end of the stream has been reached

Throws:

`IOException` - If an I/O error occurs

read

```
public int read(char[] cbuf,
                int off,
                int len)
         throws IOException
```

Reads characters into a portion of an array. This method will block until some input is available, an I/O error occurs, or the end of the stream is reached.

If `len` is zero, then no characters are read and `0` is returned; otherwise, there is an attempt to read at least one character. If no character is available because the stream is at its end, the value `-1` is returned; otherwise, at least one character is read and stored into `cbuf`.

Specified by:

`read` in class `Reader`

Parameters:

`cbuf` - Destination buffer

`off` - Offset at which to start storing characters

`len` - Maximum number of characters to read

Returns:

The number of characters read, or -1 if the end of the stream has been reached

Throws:

`IndexOutOfBoundsException` - If `off` is negative, or `len` is negative, or `len` is greater than `cbuf.length - off`

`IOException` - If an I/O error occurs

skip

```
public long skip(long n)
          throws IOException
```

Skips characters. This method will block until some characters are available, an I/O error occurs, or the end of the stream is reached. If the stream is already at its end before this method is invoked, then no characters are skipped and zero is returned.

Overrides:

`skip` in class `Reader`

Parameters:

`n` - The number of characters to skip

Returns:

The number of characters actually skipped

Throws:

`IllegalArgumentException` - If `n` is negative and the contained `Reader`'s `skip` method throws an IllegalArgumentException for a negative parameter

`IOException` - If an I/O error occurs

ready

```
public boolean ready()
              throws IOException
```

Tells whether this stream is ready to be read.

Overrides:

`ready` in class `Reader`

Returns:

True if the next read() is guaranteed not to block for input, false otherwise. Note that returning false does not guarantee that the next read will block.

Throws:

`IOException` - If an I/O error occurs

markSupported

```
public boolean markSupported()
```

Tells whether this stream supports the mark() operation.

Overrides:

`markSupported` in class `Reader`

Returns:

true if and only if this stream supports the mark operation.

mark

```
public void mark(int readAheadLimit)
          throws IOException
```

Marks the present position in the stream.

Overrides:

`mark` in class `Reader`

Parameters:

`readAheadLimit` - Limit on the number of characters that may be read while still preserving the mark. After reading this many characters, attempting to reset the stream may fail.

Throws:

`IOException` - If an I/O error occurs

reset

```
public void reset()
           throws IOException
```

Resets the stream.

Overrides:

`reset` in class `Reader`

Throws:

IOException - If an I/O error occurs

close

```
public void close()
           throws IOException
```

Description copied from class: Reader

Closes the stream and releases any system resources associated with it. Once the stream has been closed, further read(), ready(), mark(), reset(), or skip() invocations will throw an IOException. Closing a previously closed stream has no effect.

Specified by:

close in interface AutoCloseable

Specified by:

close in interface Closeable

Specified by:

close in class Reader

Throws:

IOException - If an I/O error occurs

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.